Intelligent Systems for Process Control and Automation

9 connected study units

Independent study collection

Academic context: Intelligent Systems for Process Control and Automation, Prof. Michela Mulas. These newly written learning materials are not an official course plan and do not imply the professor's authorship, review or endorsement.

Contents

1 Unit 01: Modeling, identification and simulation 3

1.1 A model begins with a boundary . . . 3

1.2 A common worked process . . . 3

1.3 Identification is an experiment-design problem . . . 4

1.4 Evidence and interpretation . . . 4

2 Unit 02: Feedback, stability and IMC tuning 5

2.1 Separate the plant, controller and actuator . . . 5

2.2 Derive an IMC design rather than memorize gains . . . 5

2.3 Implement the controller that was analyzed . . . 5

2.4 Compare outcomes on the same experiment . . . 6

3 Unit 03: Multiloop, multivariable, feedforward and cascade control 7

3.1 Match architecture to the physical disturbance path . . . 7

3.2 Cascade closes a faster measured loop . . . 7

3.3 Interactions require a joint view . . . 7

3.4 Compare architectures under matched conditions . . . 8

4 Unit 04: Constrained model predictive control 9

4.1 Predict several steps, apply only the first action . . . 9

4.2 One step exposes the tradeoff . . . 9

4.3 Constraints and offset have different causes . . . 9

4.4 Stability is a design property to establish . . . 10

5 Unit 05: Knowledge-based supervision and fuzzy control 11

5.1 Supervisory rules express decisions and evidence . . . 11

5.2 Fuzzy membership is a degree of compatibility . . . 11

5.3 Fuzzy control can act at several levels . . . 11

6 Unit 06: Neural modeling and control of dynamic systems 12

6.1 Predict a dynamic relation, not a shuffled collection of rows . . . 12
6.2 Four network families address different tasks . . . 12
6.3 Accuracy is a property of the validation experiment . . . 12
6.4 Use a learned model in control with explicit safeguards . . . 13

7 Unit 07: Evolutionary optimization and controller tuning 14

7.1 Encode a controller and an experiment together . . . 14
7.2 Reward what matters and reject invalid trials . . . 14
7.3 Separate tuning scenarios from final evaluation . . . 14

8 Unit 08: Integration, benchmark evidence and technical communication 15

8.1 A comparison is a controlled experiment . . . 15
8.2 Match the article's model identity before its score . . . 15
8.3 Evaluate performance and limitations jointly . . . 15
8.4 Communicate a reviewable conclusion . . . 15

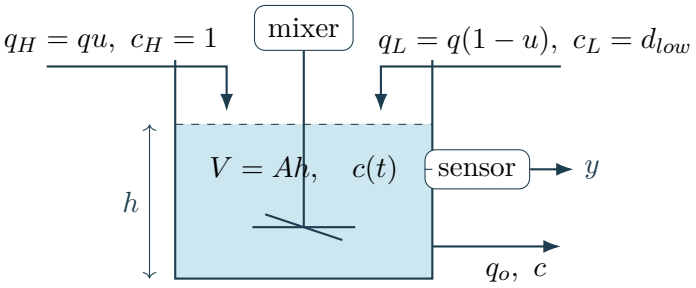
9 Unit 09: Markov chains and hidden-state diagnosis 17

9.1 An extension from process dynamics to uncertain sensor health . . . 17
9.2 From conditional probability to a Markov chain . . . 17
9.3 Long-run occupancy and time to failure are different quantities . . . 18
9.4 An HMM connects the hidden regime with observable evidence . . . 18
9.5 Filtering is causal; smoothing and complete-path decoding use later data . . . 18
9.6 Estimate parameters without joining independent experiments . . . 19
9.7 Use diagnosis as evidence for supervision, then test its limitations . . . 20

10 Bibliography and reading map 21

1 Unit 01: Modeling, identification and simulation

Prerequisites: Differential equations, units, matrix least squares and elementary probability.



Physical boundary for the analytical tank. A constant-volume model requires inlet and outlet balance. Here the disturbance is low-feed concentration; the shared laboratory instead uses an additive disturbance to the complete inlet mixture. The sensor reading may have dynamics distinct from concentration.

1.1 A model begins with a boundary

A controller changes a physical input; its software output is not automatically that input. Draw the vessel, inlet streams, outlet, sensor and valve before naming states. For a perfectly mixed tank with constant density and cross-sectional area A, conservation gives

$$A\dot{h} = q_H + q_L - q_o, \qquad \frac{d(Ahc)}{dt} = q_H c_H + q_L c_L - q_o c.$$

Expanding the derivative and substituting the volume balance cancels the outlet term in the concentration equation:

$$\dot{c} = \frac{q_H(c_H - c) + q_L(c_L - c)}{Ah}.$$

This cancellation does not mean the outlet is irrelevant: it changes h, which changes the residence time. Mixing is assumed instantaneous; density differences, reaction, evaporation and sensor dynamics are absent. A simulation approaching $h = 0$ must stop or change its physical model rather than divide by a clipped volume silently.

1.2 A common worked process

The exercises use hypothetical values $V = 100$ L, total flow $q = 10$ L/min and normalized feed concentrations $c_H = 1$, $c_L = 0$. A low-level flow system holds q constant while $q_H = qu$ and $q_L = q(1-u)$, with $0 \le u \le 1$. Thus

$$\tau\dot{c} = u - c, \qquad \tau = V/q = 10 \text{ min}.$$

These teaching parameters define the hypothetical process used in this unit. At a constant input, $c^* = u^*$. In deviation variables, $G(s) = 1/(10s+1)$. At a one-minute sampling period with a zero-order-held input,

$$c_{k+1} = ac_k + bu_k, \qquad a = e^{-0.1} = 0.904837418, \quad b = 1 - a = 0.095162582.$$

The analytical feed-composition problems use $d = d_{low}$ for the concentration of the low feed, giving an inlet mixture $u + (1-u)d_{low}$. The shared executable laboratory scenario instead uses an additive disturbance to the total inlet mixture, $u + d_{mix}$. Their conversion is $d_{mix} = (1-u)d_{low}$, so they are not interchangeable at fixed numerical values. Feedforward in the additive scenario compensates by $-d_{mix}$; the low-feed examples derive the nonlinear blend formula explicitly. Check the disturbance definition before comparing a worked answer with a laboratory run.

The exact discretization preserves the interval $[0, 1]$ because the next concentration is a convex combination of the current concentration and the inlet blend. Forward Euler gives $a_E = 1 - T_s/\tau$; it is asymptotically stable for $0 < T_s < 2\tau$ but preserves this physical interval only for $0 < T_s \leq \tau$.

1.3 Identification is an experiment-design problem

Fit $y_{k+1} = ay_k + bu_k + e_{k+1}$ by least squares, $\hat{\theta} = (\Phi^T\Phi)^{-1}\Phi^T Y$, only when the regressor matrix has full column rank. In software, use a QR/SVD least-squares solve rather than forming the inverse. A constant equilibrium record cannot identify both coefficients. Step tests help, but a bounded multilevel or persistently exciting signal gives a broader operating region.

Split complete trajectories chronologically into training, validation and final test sets. Fit scaling parameters on training data only. A one-step prediction uses the measured previous output; free simulation recursively uses its own prediction. Report both because a small one-step error can hide unstable recursive behavior. With noisy measured regressors, ordinary least squares can be biased; instrumental variables or a noise model may be needed.

1.4 Evidence and interpretation

Check mass conservation, equilibria, convergence when the integration step is halved, and agreement with the analytical step response. Inspect residual autocorrelation and correlation with past inputs on an independent trajectory. Residual whiteness alone does not prove a correct physical model. Use the benchmark inventory to distinguish equation-based simulations from experimental datasets, and do not compare numerical scores obtained with different input protocols as if they were the same experiment.

2 Unit 02: Feedback, stability and IMC tuning

Prerequisites: Unit 01; Laplace transforms, characteristic polynomials and negative feedback.

2.1 Separate the plant, controller and actuator

For the hypothetical tank $P(s) = 1/(\tau s+1)$, define the tracking error $e = r - y$ and a PI command $v = K_p e + I$, with $\dot{I} = K_i e$. The applied blend is $u = \mathrm{sat}_{[0,1]}(v)$. A control diagram that omits this saturation cannot explain windup. For a PID controller, differentiate a filtered measurement rather than an abrupt reference when derivative kick is undesirable; for example use $-K_d s/(1 + T_f s)$ on y.

With ideal measurement, no saturation and PI feedback, the closed-loop characteristic polynomial is

$$\tau s^2 + (1 + K_p)s + K_i.$$

The second-order continuous model is asymptotically stable when $\tau > 0$, $1 + K_p > 0$ and $K_i > 0$. This result does not cover delays, sampled implementation or saturating nonlinear dynamics. The final-value theorem also requires a stable closed loop before it can justify zero constant-reference error.

2.2 Derive an IMC design rather than memorize gains

For the stable minimum-phase model $\hat{P} = 1/(\tau s + 1)$, choose $Q = (\tau s + 1)/(\lambda s + 1)$. Converting the internal-model structure to ordinary feedback gives

$$C = \frac{Q}{1 - \hat{P}Q} = \frac{\tau s + 1}{\lambda s}, \qquad K_p = \frac{\tau}{\lambda}, \quad K_i = \frac{1}{\lambda}.$$

Nominal pole-zero cancellation makes $Y/R = 1/(\lambda s + 1)$, but the canceled pole still matters to internal dynamics and mismatch. With $\tau = 10$ min and $\lambda = 5$ min, $K_p = 2$ and $K_i = 0.2$ per minute. For a first-order-plus-dead-time approximation, the commonly used IMC-PI rule $K_p = \tau/[K(\lambda + \theta)]$, $T_i = \tau$ trades speed for delay tolerance; it is an approximation, not exact cancellation of a delay.

2.3 Implement the controller that was analyzed

One explicit digital update is $v_k = K_p e_k + I_k$, $u_k = \mathrm{sat}(v_k)$, then $I_{k+1} = I_k + T_s[K_i e_k + K_{aw}(u_k - v_k)]$. Here K_{aw} has units of inverse time because I has actuator units. Conditional integration is another option: freeze integration when saturation and the error would push farther into the limit. Keep update order and derivative state explicit in MATLAB, Octave and Python.

The local discrete closed-loop matrix for the unsaturated explicit update and zero reference is

$$M = \begin{bmatrix} a - bK_p & b \\ -T_s K_i & 1 \end{bmatrix}.$$

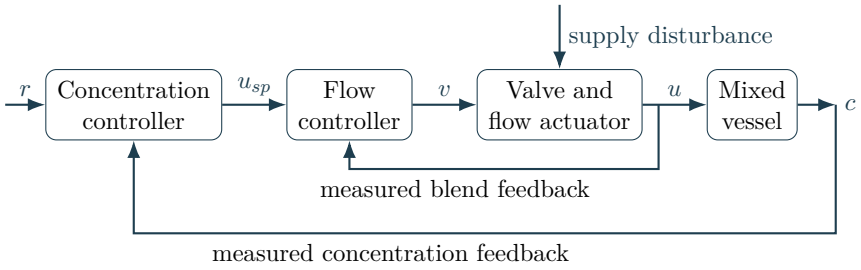
Its eigenvalues, not continuous poles alone, determine sampled stability. A reference beyond the attainable concentration range has no feasible equilibrium, however stable the local model is.

2.4 Compare outcomes on the same experiment

Use identical initial state, reference steps, feed disturbances and actuator limits. Record tracking error, settling definition, total variation $\sum |u_k - u_{k-1}|$, saturation time and any violation. A smoother input may be valuable even if its integral absolute error is larger. Disturbance rejection and sensor-noise amplification are distinct design goals; one appealing tracking plot cannot demonstrate both.

3 Unit 03: Multiloop, multivariable, feedforward and cascade control

Prerequisites: Units 01-02; matrix inversion and local transfer functions.



Cascade uses a measured inner variable. Both feedback paths are negative under the error conventions stated in the text.

3.1 Match architecture to the physical disturbance path

Feedback acts after an output error is visible. Feedforward uses a measured disturbance before that error develops. For $10\dot{c} = u + (1-u)d - c$, holding the equilibrium at r requires

$$u_{ff} = \frac{r-d}{1-d}, \qquad d \neq 1.$$

For $0 \leq d < 1$, this is feasible only when $d \leq r \leq 1$. Feedback can trim errors in d, mixing and actuator calibration; it cannot repair an impossible target. The linear cancellation formula is $F(s) = -G_d(s)/G_u(s)$. A nonminimum-phase zero or unfavorable delay in G_u can make the ideal inverse unstable or noncausal, so practical feedforward often uses a filtered approximation.

3.2 Cascade closes a faster measured loop

Suppose a valve command v produces the actual blend u through $\tau_v \dot{u} = v - u + w$, where w is a valve/load disturbance. A fast inner controller regulates measured u to an outer command u_{sp}; the outer concentration controller then sees the closed inner dynamics. With inner proportional gain K_f and the feedforward equilibrium command $v = u_{sp} + K_f(u_{sp} - u)$, the inner pole is $-(1+K_f)/\tau_v$. A practical cascade needs an inner sensor, useful bandwidth separation and consistent actuator limits. A fast model is not evidence that an actual noisy valve can move arbitrarily quickly.

3.3 Interactions require a joint view

For a local steady two-input/two-output gain matrix K, the relative gain array is $\Lambda = K \circ (K^{-1})^T$, where $\circ$ denotes elementwise multiplication. For

$$K = \begin{bmatrix} 1 & 0.6 \\ 0.4 & 1 \end{bmatrix}, \qquad \Lambda = \begin{bmatrix} 1.315789 & -0.315789 \\ -0.315789 & 1.315789 \end{bmatrix}.$$

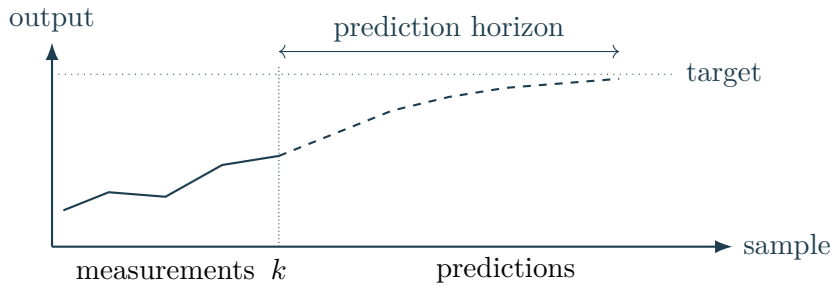
Diagonal pairing avoids negative relative gains here. RGA is a local steady-state diagnostic, not a stability proof: dynamics, delays and operating-point changes still matter. Static decoupling $u = K^{-1}v$ removes steady interaction for the nominal model, but can demand infeasible or rapidly changing physical inputs. If saturation occurs, the assumed decoupling is lost.

3.4 Compare architectures under matched conditions

Start with independently tuned PI loops, then add measured-disturbance feedforward, cascade or decoupling for a stated reason. Keep the total available measurements explicit; a design with an additional disturbance sensor has an information advantage. Introduce one disturbance at a time, then simultaneous changes and model mismatch. Report output-by-output errors as well as a combined score. In Simulink, keep the physical plant, measurement path and controller subsystems separate so their responsibilities remain visible.

4 Unit 04: Constrained model predictive control

Prerequisites: Units 01-03; quadratic forms, least squares and constrained optimization.



The future curve is a model prediction, not an observed trajectory. Only the first planned input is applied before the next measurement.

4.1 Predict several steps, apply only the first action

For $x_{k+1} = Ax_k + Bu_k$, $y_k = Cx_k$, assemble predicted outputs as $Y = Fx_k + GU$, where $U = [u_k^T, \ldots, u_{k+N-1}^T]^T$. The first block rows are CA in F and CB in G; the second are CA^2 and $[CAB, CB]$. Use a consistent convention: these predictions begin at $k+1$.

A useful objective is

$$J = (Y - R)^T Q(Y - R) + (DU - d_0)^T R_u(DU - d_0),$$

where $DU - d_0$ stacks input increments and its first element is $u_k - u_{k-1}$. Bounds on U, input increments and predicted outputs become linear inequalities. Penalizing input magnitude instead of increments solves a different problem, especially around nonzero operating points. The Hessian is positive definite when the regularization supplies strictly positive curvature in every free input direction.

At each sample, estimate the state, solve the optimization, apply the first action, and repeat. Use a shorter control horizon by holding the last planned move constant; write this mapping explicitly rather than silently changing the prediction length. Log solver status, time and fallback use. A failed solver must not be treated as a valid optimum.

4.2 One step exposes the tradeoff

For $c^+ = ac + bu$ and $J = (c^+ - r)^2 + \rho(u - u_{old})^2$, the unconstrained optimum is

$$u^* = \frac{b(r - ac) + \rho u_{old}}{b^2 + \rho}.$$

With only scalar interval constraints, project this value onto their intersection. This is exact for the scalar one-step problem; independently clipping a general coupled multistep solution is not equivalent to solving its constrained quadratic program.

4.3 Constraints and offset have different causes

If the target lies beyond the attainable set, no horizon removes that incompatibility. Soften selected output targets with nonnegative slack and an explicit penalty when the application permits it; keep

actuator bounds hard. For persistent model mismatch, augment a constant disturbance state and estimate it, or add another justified offset-free mechanism. A large prediction horizon alone does not guarantee zero steady error.

4.4 Stability is a design property to establish

Finite-horizon optimization and decreasing simulated error do not automatically imply closed-loop stability. Classical sufficient conditions involve suitable terminal cost, terminal invariant constraints and a feasible terminal controller. The teaching implementations demonstrate finite-horizon behavior under their declared experiments; do not infer a global nonlinear stability theorem. For multivariable processes, inspect individual output constraints and input allocation, not just the sum of squared errors.

5 Unit 05: Knowledge-based supervision and fuzzy control

Prerequisites: Units 01-03; sets, piecewise functions and numerical integration.

5.1 Supervisory rules express decisions and evidence

A rule such as IF the concentration residual is large AND the flow sensor agrees with its command THEN suspect a concentration-path fault represents a diagnostic hypothesis. It does not prove a failed sensor. Store the measured evidence, persistence interval, action and reset condition separately. Use priorities for conflicting rules: an invalid measurement should inhibit a normal optimization request. Hysteresis and dwell time prevent repeated switching near a threshold. A low concentration alone cannot distinguish an inlet change, valve fault or concentration-sensor bias.

5.2 Fuzzy membership is a degree of compatibility

On a normalized error domain $[-1, 1]$, define negative, zero and positive memberships by $\mu_N(e) = \max(0, -e)$, $\mu_Z(e) = 1 - |e|$ and $\mu_P(e) = \max(0, e)$, after explicitly limiting e to the domain. They sum to one. These numbers are not probabilities unless a separate probabilistic interpretation is established. Input scaling determines which physical errors activate each rule.

For a two-input controller using error and error change, a rule might be IF error is positive AND error change is positive THEN increase the blend strongly. Specify whether AND is minimum or product. Mamdani inference clips or scales an output membership function for every firing rule, aggregates the results, and often defuzzifies with the centroid:

$$u_f = \frac{\int z\, \mu_{out}(z)\, dz}{\int \mu_{out}(z)\, dz}.$$

For zero-order Sugeno inference, singleton consequents z_i give $u_f = (\sum_i w_i z_i)/(\sum_i w_i)$. This generally differs from the centroid of overlapping Mamdani sets. If no rule fires, use a documented fallback; adding an arbitrary epsilon to the denominator does not define a meaningful inference result.

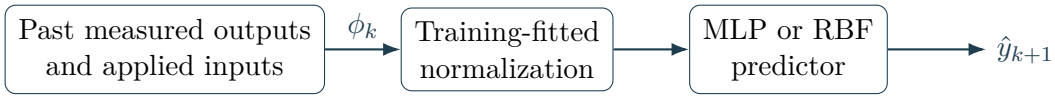
5.3 Fuzzy control can act at several levels

A fuzzy controller may directly choose the command, produce its increment, or schedule PI/PID gains. If $u_k = u_{k-1} + K_u u_f$, then K_u and the sampling interval determine how quickly the actuator moves. A fuzzy PID schedule changes K_p, K_i, K_d with operating conditions; integrate with the current gain convention consistently and retain anti-windup. A stable PI at every fixed gain setting does not by itself prove stability during arbitrary gain switching.

For the tank, positive concentration error should generally increase the high-feed fraction. The rule surface must respect this known sign. Examine its monotonicity, continuity and saturated behavior before closing the loop. Compare against the same PI baseline with identical sensors and limits. Display both the membership functions and the resulting control surface; a list of linguistic rules alone is insufficient for reproduction.

6 Unit 06: Neural modeling and control of dynamic systems

Prerequisites: Units 01, 02 and 04; differentiation, gradients, least squares and separation of training, validation and test data.



Causal regressors and fixed preprocessing are part of the model. Recursive simulation substitutes previous predictions for measured outputs; it is a different validation task.

6.1 Predict a dynamic relation, not a shuffled collection of rows

A nonlinear autoregressive model with external input uses past observations and applied inputs to predict the next output. One causal choice is

$$\phi_k = [y_k, y_{k-1}, u_k, u_{k-1}]^T, \qquad \hat{y}_{k+1} = f_\theta(\phi_k).$$

The regressor timing must be causal. Randomly shuffling heavily overlapping windows before splitting can place nearly identical trajectories in both training and testing. Split trajectories first, then construct lagged windows inside each partition. Save scaling parameters, random seed, architecture, weights and the complete data protocol.

6.2 Four network families address different tasks

A perceptron computes a binary decision from $\text{sign}(w^T x+b)$ and updates the separating hyperplane on classification mistakes. It is a useful diagnostic classifier for linearly separable conditions, not a continuous process simulator. An MLP composes affine maps and nonlinearities; for one hidden layer, $h = \tanh(W_1\phi + b_1)$ and $\hat{y} = W_2 h + b_2$. Backpropagation differentiates the loss through both layers. Regularization, early stopping and independent validation limit overfitting but do not certify extrapolation.

An RBF network uses local responses $\psi_j(\phi) = \exp[-\|\phi-\mu_j\|^2/(2\sigma_j^2)]$ and a linear output. For fixed centers/widths, ridge regression determines output weights from $(\Psi^T\Psi + \alpha I)w = \Psi^T Y$. Poorly placed centers can leave important operating regions unsupported. A self-organizing map (SOM) updates a best-matching unit and its neighbors, $m_j^+ = m_j + \eta h_{j,j^*}(x - m_j)$. It organizes operating regimes without supervised target values; a small map distance is not a fault label or a probability.

6.3 Accuracy is a property of the validation experiment

Report one-step and recursive simulation errors, operating-range coverage, residual structure and behavior under changed initial conditions. Normalize errors with a declared physical scale. Preserve the physical concentration interval and compare against the linear identification baseline: a larger network is useful only if it answers a limitation the baseline actually has. Bootstrap or repeated-seed comparisons should resample whole independent runs where temporal correlation matters.

6.4 Use a learned model in control with explicit safeguards

A neural predictor can replace a prediction model inside MPC, or a differentiable inverse can propose an input. For a locally monotone model, a bounded Newton step is $u^+ = u - \gamma(\hat{y}^+(u) - r)/(\partial\hat{y}^+/\partial u)$. A small or wrong-sign derivative makes this unreliable. Enforce actuator limits, restrict the search to supported inputs, test the candidate against a known model/constraint layer, and retain a validated fallback. Direct inverse learning can be ill-posed when different inputs produce the same output or when unobserved states matter. Good training error does not establish closed-loop stability.

7 Unit 07: Evolutionary optimization and controller tuning

Prerequisites: Units 02, 04 and 06; random sampling, objective functions and constrained experiment design.

7.1 Encode a controller and an experiment together

A genetic algorithm searches a population of candidate parameters through selection, crossover and mutation. For PI tuning the chromosome may encode (K_p, K_i); for multivariable MPC it may encode prediction horizon and positive input/output weights. Bounds and units are part of the representation. A chromosome is not a controller until the decoding, simulation protocol, constraints and cost are defined.

For a b-bit unsigned integer q, a parameter in $[l, h]$ is $\theta = l + (h - l)q/(2^b - 1)$. Binary encoding creates quantization and bit-level neighborhoods that may differ strongly from physical distance. Real encoding avoids that quantization but still needs a documented crossover and mutation distribution. Blend crossover $\theta_c = \alpha\theta_1 + (1 - \alpha)\theta_2$ stays inside the parental interval when $0 \leq \alpha \leq 1$; Gaussian mutation may leave bounds, so repair or rejection must be specified.

7.2 Reward what matters and reject invalid trials

One cost is $J = \text{IAE} + \beta \sum_k |u_k - u_{k-1}| + \gamma \sum_k \epsilon_k^2$, where ϵ measures permitted soft violations. Normalize terms or state their units before choosing weights. Hard physical limits should be enforced or treated as candidate rejection, not hidden by an average penalty. Terminate unstable simulations using declared state/finite-value thresholds and give them a consistently poor fitness.

Tournament selection needs only an ordering, whereas fitness-proportional selection needs nonnegative appropriately scaled fitness. Elitism preserves the best observed valid candidate; it does not establish a global optimum. Count simulation evaluations, not just generations, when comparing computational budgets. Include a grid, random search or established IMC tuning baseline so improvement can be attributed to the search method rather than additional evaluation effort.

7.3 Separate tuning scenarios from final evaluation

Optimize against a fixed family of training trajectories, select algorithm settings using validation scenarios, then freeze the design before testing unseen disturbances and plant mismatch. Repeated seeds reveal stochastic search variability; report the distribution and all failed runs, not only the best seed. For uncertain models, use a declared robust objective such as the maximum cost across a finite scenario set; this is robustness to that tested set, not a proof for all uncertainty.

For multivariable MPC, chromosomes must retain positive weights and integer horizons. Changing a weight changes both performance and the meaning of a reported objective; compare final controllers using a common external evaluation metric. Include solve-time and feasibility effects because a long horizon that misses the deadline is not an operational improvement.

8 Unit 08: Integration, benchmark evidence and technical communication

Prerequisites: Units 01-07; interpretation of numerical experiments and model limitations.

8.1 A comparison is a controlled experiment

Start by stating the plant, operating range, measured variables, actuator limits and decision question. Establish the physical and linear identification baselines before adding neural or fuzzy components. Freeze a shared experiment contract: sample interval, numerical solver and tolerance, initial state, reference, disturbances, noise realization, controller initialization and failure policy. Keep synthetic outputs distinct from measured records. A simulation-based controller study is evidence about its declared model and scenarios.

Use the supplied integrated syllabus as a non-official study framework. The three proposed practical blocks become one connected project: identify and tune a baseline; compare alternative structures and MPC; then add an intelligent method and test its incremental value. Do not give every method a different experiment merely because one plot looks better.

8.2 Match the article's model identity before its score

Kroll and Schulte's article provides a benchmark survey, not a single self-contained software package. Its sections include artificial SISO and MIMO identification models; data repositories; an inverted pendulum; throttle; pneumatic actuation; hydrostatic transmission; three tanks; a bioreactor; a chemical CSTR; a binary distillation column; the ALSTOM gasifier; wind-turbine models; and the Tennessee Eastman process. Some examples specify equations; others require external data, maps, parameter choices or full reference simulators.

Record the section, equation/model variant, source version, parameters, units and any closure added by the implementation. An educational closure can illuminate a mechanism, but its results do not reproduce an external industrial benchmark. Missing physical functions must remain a visible dependency rather than be replaced by an arbitrary function with the original benchmark's name. The repository's benchmark inventory is the authority for implemented versus externally dependent cases.

8.3 Evaluate performance and limitations jointly

Prediction metrics include maximum absolute error, SSE, RMSE, VAF and R^2. VAF uses residual variance while R^2 uses residual squared magnitude, so a constant bias can leave VAF deceptively high. For constant targets, denominator-based metrics may be undefined; report absolute errors rather than inventing a finite score. Control metrics include IAE/ISE, peak error, settling under a stated band/dwell rule, applied effort, total variation, saturation and violations. Analyze trajectories as well as aggregates.

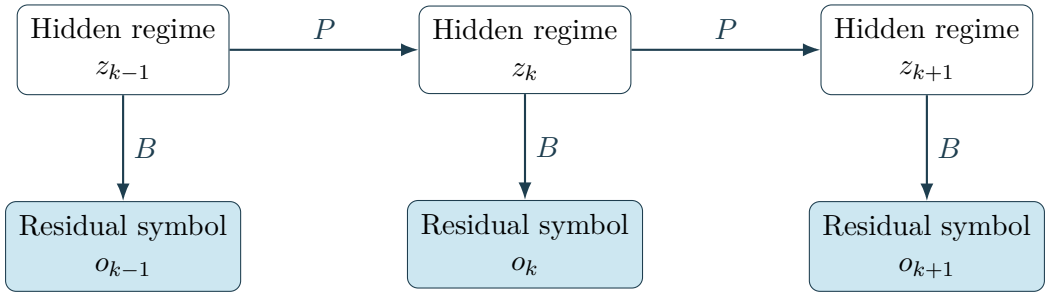
8.4 Communicate a reviewable conclusion

A technical report should connect requirements, derivation, implementation, experiment, result and limitation. Include a physical equipment diagram, a control architecture diagram and axes with units. Give enough code and configuration to reproduce every reported value. State which language/runtime was actually executed. An unrun MATLAB or Simulink port must not be presented as experimentally verified merely because Python passed.

Present improvements as conditional results: for this model, this information set and these tests, the observed tradeoff was favorable. Explain failures and unsupported extrapolation. A useful next decision may be to collect a missing measurement or validate a plant parameter, rather than introduce another algorithm.

9 Unit 09: Markov chains and hidden-state diagnosis

Prerequisites: Units 01, 02 and 08; conditional probability, matrix multiplication and elementary statistics.



Transitions describe regime persistence; emissions connect a regime to uncertain evidence. At sample k, an online filter can use observations only through o_k.

9.1 An extension from process dynamics to uncertain sensor health

This additional unit extends the eight content areas of the supplied syllabus. Markov chains and hidden Markov models (HMMs) provide a probabilistic diagnosis exercise built around the teaching tank; this is not a model attributed to the 2014 benchmark article or a claim about the professor's official syllabus. The mathematical reading is Grinstead and Snell, Chapter 11, and Rabiner's HMM tutorial, Sections II-III and V, listed as references 14 and 15.

The physical concentration and the sensor's condition answer different questions. The familiar nominal tank has $V = 100$ L, $q = 10$ L/min and residence time $\tau = 10$ min. Its sampled balance remains $c_{k+1} = ac_k + (1 - a)u_k$, now with $T_s = 0.2$ min and $a = e^{-0.02}$. A discrete regime z_k describes normal operation, drift or a sensor fault. An HMM estimates that regime from measurements; it does not replace the mass balance or make concentration a three-valued variable.

In the synthetic laboratory, $y_k = c_k + b_{z_k} + \epsilon_k$, with biases $b = [0, 0.055, 0.14]$ and independent Gaussian noise of standard deviation 0.035 in normalized concentration. Form a residual $r_k = y_k - \hat{c}_k$, where the nominal model prediction uses only the initial condition and previously applied inputs. Assign symbols 0, 1 and 2 using cut points 0.025 and 0.095: symbol 0 for $r_k < 0.025$, symbol 1 for $0.025 \le r_k < 0.095$, and symbol 2 otherwise. These bins overlap in their possible hidden causes because the noise is nonzero. A large residual is evidence, not a directly observed fault label.

9.2 From conditional probability to a Markov chain

For events with positive conditioning probability, $\Pr(A \mid B) = \Pr(A \cap B)/\Pr(B)$. A first-order Markov model assumes that the present regime contains the information from its history needed to predict the next regime:

$$\Pr(z_{k+1} = j \mid z_k = i, z_{k-1}, \ldots, z_0) = \Pr(z_{k+1} = j \mid z_k = i) = P_{ij}.$$

This does not make successive states independent. Time homogeneity additionally assumes that the same P applies at every sample. Here distributions are row vectors, rows index the current state and columns index the next state. Nonnegative entries and row sums of one imply $p_{k+1} = p_k P$ and $p_{k+n} = p_k P^n$. Matrix multiplication sums probabilities over intermediate states; do not raise each entry individually to the nth power.

The laboratory uses the state order normal, drift, sensor fault and the following deliberately hypothetical probabilities:

$$P = \begin{bmatrix} 0.97 & 0.025 & 0.005 \\ 0.04 & 0.93 & 0.03 \\ 0.02 & 0.08 & 0.90 \end{bmatrix}, \qquad p_0 = [0.80, 0.15, 0.05].$$

The matrix allows recovery from a fault. Its entries describe one 0.2-minute step, not failure rates per minute. To sample the same discrete model every five steps, use P^5; changing to a finer interval requires a justified model, and an arbitrary matrix root need not be a transition matrix. Physical residence time and regime persistence are distinct parameters. A self-transition probability $P_{ii} < 1$ produces a geometric consecutive residence duration with mean $1/(1 - P_{ii})$ samples, counted from entry through the last sample in state i.

9.3 Long-run occupancy and time to failure are different quantities

A stationary row distribution satisfies $\pi P = \pi$ and $\sum_i \pi_i = 1$. A finite irreducible chain has a unique stationary distribution; aperiodicity additionally ensures convergence of $p_0 P^n$ to it from every initial distribution. Check these conditions rather than interpreting any numerical eigenvector as a unique limiting probability. A two-state chain that always alternates has a stationary distribution but its marginal probabilities can oscillate forever.

An absorbing fault model answers a different question. Partition the states into transient and absorbing groups so $P = \left[\begin{smallmatrix} Q & R \\ 0 & I \end{smallmatrix}\right]$. If every transient state reaches the absorbing set with probability one, the fundamental matrix is $N = (I-Q)^{-1}$ and its row sums give expected steps until absorption. Compute the times by solving $(I - Q)t = \mathbf{1}$, then multiply by T_s for minutes. The derivation is the first-step relation $t_i = 1 + \sum_j Q_{ij} t_j$. If an unreachable closed class exists, the expected time from some states is infinite and this finite solve is not valid. With one absorbing fault and an initial transient row q_0, the probability of absorption within n steps is $1 - q_0 Q^n \mathbf{1}$. An expected lifetime is neither a deadline nor a guarantee about one trajectory.

9.4 An HMM connects the hidden regime with observable evidence

For a categorical HMM, $B_{is} = \Pr(o_k = s \mid z_k = i)$ is the emission matrix, with one normalized row per hidden state. The model assumes conditional independence of emissions given the state sequence. Its initial distribution, transition matrix and emissions define a joint probability:

$$\Pr(z_{0:T-1}, o_{0:T-1}) = p_0(z_0) B_{z_0 o_0} \prod_{k=1}^{T-1} P_{z_{k-1} z_k} B_{z_k o_k}.$$

The first observation is emitted by z_0 before a transition. This indexing convention must agree across simulation, filtering and estimation. The training data contain synthetic state labels, allowing emission probabilities to be learned from counts. Labels are available for evaluation but are withheld from the online estimator and controller. For a real experiment, maintenance records or an independent calibration procedure would be needed to justify such labels. Correlated noise, an unmeasured process disturbance or an incorrect tank model can violate the simple emission assumption and be confused with a sensor defect.

9.5 Filtering is causal; smoothing and complete-path decoding use later data

Let $\alpha_k(i) = \Pr(z_k = i \mid o_{0:k})$. Initialize with $\alpha_0 \propto p_0 \odot B_{:,o_0}$. At later samples, predict $\bar{\alpha}_k = \alpha_{k-1} P$, multiply by the current emission likelihood and normalize:

$$w_k(i) = \bar{\alpha}_k(i) B_{i,o_k}, \qquad s_k = \sum_i w_k(i), \qquad \alpha_k(i) = w_k(i)/s_k.$$

Use $\bar{\alpha}_0 = p_0$ for the initial update. Scaling at every step avoids the underflow produced by multiplying a long sequence of small probabilities. With this definition of s_k, the sequence log likelihood is $\sum_k \log s_k$. Some references use reciprocal scale factors and therefore a negative sum; declare the convention. A zero scale signals an observation sequence impossible under the model, not permission to replace it silently with a uniform posterior.

For offline smoothing, initialize $\beta_{T-1}(i) = 1$ and run backwards:

$$\beta_k(i) = \frac{\sum_j P_{ij} B_{j,o_{k+1}} \beta_{k+1}(j)}{s_{k+1}}.$$

Normalize $\alpha_k \odot \beta_k$ to obtain $\gamma_k(i) = \Pr(z_k = i \mid o_{0:T-1})$. The final smoothed posterior equals the final filtered posterior, but earlier values can change when future observations arrive.

Viterbi decoding finds the single most probable complete state path, using maximization, predecessor pointers and a final backward trace. Filtering and smoothing instead sum over alternative paths. Implement Viterbi in log space, treating a zero probability as negative infinity. Taking the largest marginal probability at each time generally does not maximize the joint path probability. All three recursions cost order Tm^2 for T observations and m states; exhaustive path enumeration is useful only for small verification examples. Smoothing and full-record Viterbi may be evaluated offline, but feeding their earlier estimates to an online controller leaks future observations.

9.6 Estimate parameters without joining independent experiments

For labelled training sequences, let C_{ij} count transitions within each sequence and E_{is} count state-symbol pairs. With pseudocount $\eta > 0$, $\hat{P}_{ij} = (C_{ij} + \eta)/(\sum_j C_{ij} + m\eta)$; normalize emission counts in the same way using the number of symbols. Estimate the initial distribution from the first state of each sequence. Never count a transition from the end of one experiment to the start of another. The laboratory uses 20 separate training trajectories of 150 samples, seed 17, and a separate 600-sample evaluation trajectory, seed 2026, with $\eta = 1$. Bins, bias values, model structure and alarm policy are fixed before test evaluation.

When state labels are unavailable, Baum-Welch alternates forward-backward inference with expected-count updates. The expected occupancy is $\gamma_k(i)$; the expected transition count at a sample is proportional to $\alpha_k(i) P_{ij} B_{j,o_{k+1}} \beta_{k+1}(j)$. Normalize these pair probabilities, sum within trajectories and update P, B and the initial distribution. Transition denominators exclude each final sample; emission denominators include it. Exact unregularized EM does not decrease training likelihood, but may find a local optimum and is sensitive to initialization. A larger training likelihood does not establish better fault detection. Hidden-state labels can permute without changing likelihood; assigning the words normal, drift and fault requires separate semantic evidence. Preserve the distinction between an unsupervised fit and labelled validation.

The optional laboratory EM demonstration starts from the known simulator transition matrix, initial distribution and theoretical emission probabilities obtained from its Gaussian noise and bin boundaries. This deliberately favorable synthetic initialization limits conclusions to parameter refinement under that information advantage. It supplies no evidence of discovering meaningful fault regimes from an uninformed initialization. Keep the labelled supervised diagnostic model separate from this training-likelihood demonstration.

9.7 Use diagnosis as evidence for supervision, then test its limitations

The example alarm uses $\alpha_k(\text{fault}) \geq 0.7$. Estimate a bias $\hat{b}_i$ for each labelled regime from its mean residual in the training data. A separate demonstration subtracts the posterior expected sensor bias, $\hat{b}_k = \sum_i \alpha_k(i)\hat{b}_i$, before passing $y_k - \hat{b}_k$ to the PI loop. The online controller receives these fitted bias estimates; the generator's true biases b_i remain part of the synthetic experiment definition. Both decisions use filtered probabilities available at sample k; neither receives the current true state. Compare with the same PI loop using the raw measurement, the same hidden-state and noise sequences, and the same input constraints. An alarm threshold is an explicit operating choice, not a universal property of an HMM. False alarms, missed faults, fault-episode detection delay, posterior calibration, tracking error and actuator effort describe different tradeoffs.

Stress the diagnosis by changing the actual residence time to 14 min while retaining the 10-minute nominal predictor. The resulting model residual can resemble bias even when the sensor is healthy. Report this confusion, noisy transient classifications and any control degradation rather than assuming that bias compensation must help. This laboratory is a synthetic supervisory experiment. It supplies no formal stability, reliability or safety guarantee, and it does not validate a physical instrument or replace an independently designed protection layer.

10 Bibliography and reading map

The first twelve entries reproduce the bibliographic scope of the supplied integrated syllabus. Years/editions follow that source; they are not a claim that every chapter or edition was consulted. The article supplies the benchmark framework. Entries 14 and 15 support the additional Markov/HMM unit, which extends that scope.

1. Massa de Campos, M.; Saito, K. *Sistemas inteligentes em controle e automação de processos*. Ciência Moderna, 2004.

2. Nascimento Jr., C. L.; Yoneyama, T. *Inteligência artificial em controle e automação*. Blücher, 2000.

3. Nørgaard, M.; Ravn, O.; Poulsen, N. K.; Hansen, L. K. *Neural Networks for Modelling and Control of Dynamic Systems*. Springer, 2009.

4. Passino, K.; Yurkovich, S. *Fuzzy Control*. Pearson, 1997.

5. Vidal, Leonardo de Carvalho. *Aplicação Fuzzy x PID: Técnicas de Controle em Ambientes Simulados*. 1st ed., Appris, 2015. ISBN 978-8581927800.

6. Aguirre, Luis Antônio. *Enciclopédia de Automática*, vol. 1. Blücher, 2007.

7. Aguirre, Luis Antônio. *Enciclopédia de Automática*, vol. 2. Blücher, 2007.

8. Camacho, E.; Bordons Alba, C. *Model Predictive Control*. Springer, 2004.

9. Ljung, L. *System Identification: Theory for the User*. Pearson, 1998.

10. Simões, Marcelo Godoy; Shaw, Ian. *Controle e modelagem fuzzy*. 2nd ed., Blücher, 2007. ISBN 978-8521204169.

11. Skogestad, Sigurd; Postlethwaite, Ian. *Multivariable Feedback Control: Analysis and Design*. 2nd ed., Wiley, 2005.

12. Tuovinen, T.; Périaux, J.; Neittaanmäki, P. (eds.). *Computational Sciences and Artificial Intelligence in Industry: New Digital Technologies for Solving Future Societal and Economical Challenges*. Springer, 2022.

13. Kroll, A.; Schulte, H. Benchmark problems for nonlinear system identification and control using Soft Computing methods: Need and overview. *Applied Soft Computing* 25 (2014), 496-513. DOI: 10.1016/j.asoc.2014.08.034.

14. Grinstead, C. M.; Snell, J. L. *Introduction to Probability*. 2nd ed., American Mathematical Society, 1997. Chapter 11: Markov chains, absorption and stationary distributions. Author-hosted open text.

15. Rabiner, L. R. A tutorial on hidden Markov models and selected applications in speech recognition. *Proceedings of the IEEE* 77(2) (1989), 257-286. DOI: 10.1109/5.18626. Sections II-III and V: inference, learning, scaling and separate observation sequences. University-hosted article.

Suggested reading by unit: 01 (3, 9, 13); 02 (1, 6, 7); 03 (1, 11); 04 (8, 11); 05 (1, 2, 4, 5, 10); 06 (2, 3); 07 (1, 2, 12); 08 (8, 9, 11, 12, 13); 09 (14, 15).